RideMatch

Project Team

Sameer Tahir 20I-0455
Hamzah Ahmad 21I-0254
Hasnain Jafri 21K-3116

Session 2024-2025

Supervised by

Dr. Syed Qaiser Ali Shah

Department of Computer Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

October, 2024

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Problem Statement

Students face an enormous problem finding affordable, safe and socially fulfilling commuting options. The fluctuating fuel costs have made transportation more expensive, particularly for students, with limited budgets. At the same time students with social anxiety or students who want to network can't find potential people in traditional transport.

1.2 Scope

Ride-Match is a shared fair carpooling app that means to help the students socialize, connect and commute with their peers. Paired with AI technologies, Ride-Match offers its users to match users with their ideal match in terms of personality, likes, dislikes, preference and routine. The platform will decide the best route for all of the passengers in a journey to ensure quick, budget friendly and convenient travelling. Ride-Match aims to help the users decide on who they would like to travel with in the future by giving them the option to match with someone of interest by providing them a list of peers that match their social profile. The app will also provide a platform for the drivers, so the limitation of a student's availability is negated as a factor of inconvenience. For seamless transactions the platform will also provide an inbuilt wallet so handling your monthly budget would no longer be a hassle. Ride-Match will authenticate their users by mobile number verification, so each rider, passenger can be stress free in their daily travels

1.3 Modules

1.3.1 Social Profiling

Each user will have a detailed social profile that includes information about their interests, preferences, and desired travel companions.

1. This module will be achieved by information gathering at the registration and using radio buttons that will match the description of the passengers using NLP and Cosine Similarity

1.3.2 Route Matching

The app will use advanced algorithms to match passengers whose routes align, ensuring efficient travel from point A to point B, with potential stops at points C and D for other passengers along the way.

1. This module will be achieved by using Dynamic Time Warping

1.3.3 Personnel Selection

This module will display a list of potential people you would like to add in a future journey or to communicate with.

1. This module will use collaborative filtering with matching peers, mutual friends and similar interests

1.3.4 Driver Mode

This module will give an interface for the drivers to accept rides for different passengers

1.3.5 Chatting Feature With Inbuilt ChatBot

This module will allow the users to communicate with their fellow passengers, and can also communicate with the rider. An inbuilt chatbot will be used for improving interactions

1. The ChatBot will be implemented using transformers

1.3.6 User Authentication

The app will authenticate users using their mobile phone numbers

1.3.7 In-App Wallet

An integrated wallet will allow users to store credits and make seamless transactions, simplifying the payment process and enhancing the overall user experience.

1.4 User Classes and Characteristics

Below are the user classes we intend to use and their characteristics.

Table 1.1: User Classes and Characteristics

User Class	Description
Rider	Users seeking to share rides for economical and social commuting. They can create and maintain a personal profile, search for compatible rides, select drivers, engage in communication with other riders and drivers, provide ratings and feedback post-ride, and manage payments through an in-app wallet system.
Driver	Drivers are users who offer rides by listing their available routes and times. They manage ride requests, coordinate with riders, and ensure safe and efficient trips.

Chapter 2

Project Requirements

2.1 Use-case Diagram

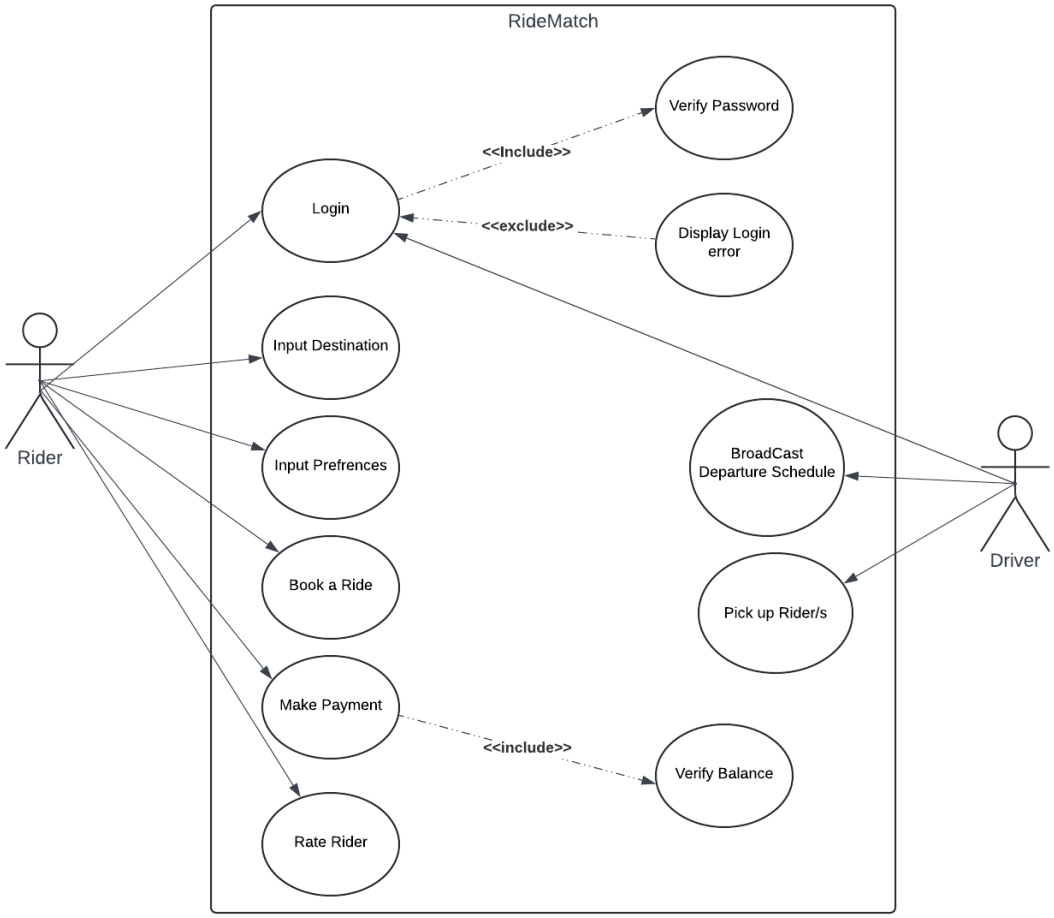

Figure 2.1: Use-Case Diagram

2.2 Functional Requirements

2.2.1 Social Profiling

2.2.1.1 User Profile Creation

- The system must allow users to input their interests, preferences, and desired characteristics of travel companions during the registration process.

2.2.1.2 Profile Matching

- The system should utilize NLP to analyze and match user profiles based on the descriptions provided, ensuring compatibility in preferences.

2.2.2 Route Matching

- The system must use Dynamic Time Warping to analyze and match routes submitted by users, finding optimal shared travel routes.

2.2.3 Personnel Selection

- The system should use collaborative filtering techniques to suggest potential ride companions based on mutual friends and interests.

2.2.4 Driver Mode

2.2.4.1 Ride Management

- The system must provide a dedicated interface for drivers to view, accept, or decline ride requests.

- Allow drivers to broadcast their future departure schedules.

2.2.5 Chatting Feature With Inbuilt ChatBot

2.2.5.1 Communication Interface

- The system must support real-time messaging among users to facilitate communication before and during rides.

2.2.5.2 ChatBot

- The system's ChatBot should assist users by providing automated responses and support.

2.2.6 User Authentication

- The system must securely authenticate users using their mobile phone numbers, ensuring that each user account is uniquely accessed.

2.2.7 In-App Wallet

2.2.7.1 Wallet Integration

- The system's in-app wallet should enable users to securely store credits and make payments for rides within the application.

2.3 Non-Functional Requirements

2.3.1 Reliability

- The system should automatically recover from a failure within 2 minutes, ensuring minimal disruption to the service.

2.3.2 Usability

2.3.2.1 Ease of Use

- The user interface should be intuitive and easy to use, allowing new users to navigate and perform essential tasks with minimal instruction.

2.3.3 Performance

2.3.3.1 Response Time

- The system should respond to user requests within 2 seconds under normal operating conditions.

2.3.3.2 Transaction Speed

- Wallet transactions should be processed and the balance must be reflected in user accounts within 4 seconds.

2.3.4 Security

- Users' data must be encrypted and resistant to data leaks.

2.3.5 Scalability

- Using AWS services, the system should scale effectively to accommodate growth in the user base, without degrading the performance of the application.

2.4 Domain Model

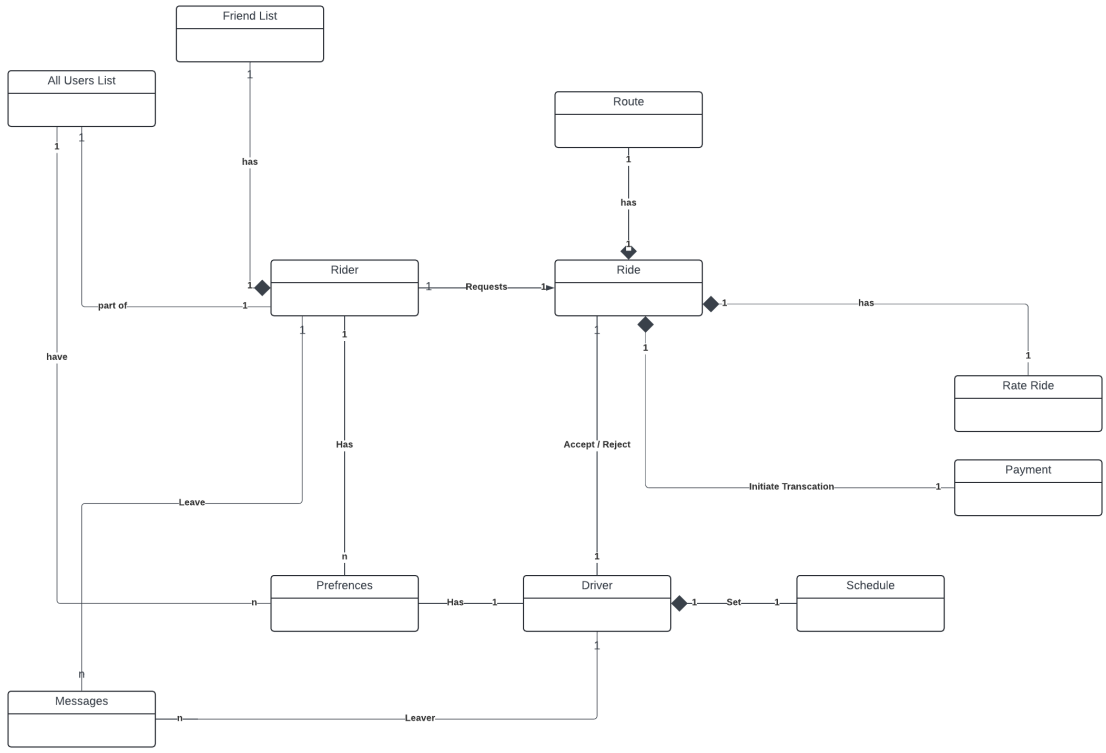

Figure 2.2: Domain Model

Chapter 3

System Overview

3.1 Architectural Design

The architectural design of RideMatch is structured to ensure modularity, scalability, and maintainability while supporting core features like ride matching, user profile management, and social connections. The system follows a client-server model, where the mobile application serves as the client and interacts with the backend for data processing, storage, and matchmaking functionalities.

3.1.1 Modular Program Structure

The system is divided into several modules, each responsible for distinct functions:

- **User Management Module:** Handles user registration, authentication, and profile management. It also stores user preferences, such as route details, car ownership status, and social interests.
- **Ride Matching Module:** The core of the system, responsible for matching users based on route proximity, travel schedules, and additional preferences like gender and social compatibility.
- **Social Matching Module:** Independent of the ride-matching functionality, this module allows users to connect based on shared interests or characteristics, enhancing campus social interactions.
- **Notification and Communication Module:** Sends real-time notifications to users regarding ride matches, social connections, and any changes in travel plans.
- **Data Storage and Management Module:** Interfaces with the database to store user data, route information, and match history. It ensures data security and integrity.

- **Backend and API Module:** The backend processes requests from the mobile client, including ride searches, user profile updates, and match confirmations. This module also manages communication with third-party services like Google Maps API.

3.1.2 System Workflow

1. **User Registration & Profile Management:** Users create accounts, input personal details, travel routes, and social preferences. These details are stored in the database.
2. **Route and Ride Matching:** The app periodically queries the backend to find ride matches based on the user's entered route and time preferences.
3. **Social Matching:** Users can explore social matches based on shared interests, independent of their ride-sharing activity.
4. **Notifications:** Matched riders or social connections receive notifications via the app.
5. **Data Storage:** All user data and activity (e.g., profile details, route, match history) are securely stored in the backend's database.

3.1.3 Data Required to Get the Model started

1. **User Data:**
   - Profile Information: Age, gender, interests, etc.
   - Preferences for Ride Partners: E.g., Others interests, gender preference, etc.
   - Ride History and Ratings: Past ride experiences, ratings, feedback. (Will be deployed once the app is functional)
2. **Ride Data:**
   - Pickup and Drop-off Locations: Latitude, longitude, and time (planned time of departure).
   - Route Data: Google Maps APIs to estimate the route overlap between users.
3. **Driver Data:**
   - Vehicle Information: Type, capacity, etc.
   - Driver's Available Time: When the driver is free for the ride.

3.1.4 Models to Use

1. **Preference Matching (Collaborative Filtering or Content-based Filtering):** We are using a recommendation system approach for Riders matching.

    - **Collaborative Filtering:** Recommends others who have been rated or selected similarly. This could be user-based or item-based collaborative filtering:
        - User-based Collaborative Filtering: Match user1 with user2 based on similar preferences from past ride history and feedback.
        - Item-based Collaborative Filtering: Compare users' profiles to find commonalities in preferences (age, gender, interests, etc.).
        - **Algorithms:**
            * Matrix Factorization: E.g., Singular Value Decomposition (SVD) for collaborative filtering.
            * k-Nearest Neighbors (k-NN)
    - **Content-based Filtering:** Match users based on the similarity of their attributes. For example, if Rider 1 prefers Cricket and Technology, Rider 2 should match those traits.
        - **Algorithms:**
            * Cosine Similarity or Euclidean Distance: These can be used to compute the similarity between user preference vectors.

2. **Route Matching (Geospatial Matching):**

    - Matching users based on the similarity of their routes requires calculating the overlap between routes.
    - **Techniques:**
        - DBSCAN: For low GPS sampling rate.
        - Dynamic Time Warping (DTW): This technique could be used for comparing sequences of geographic points (routes) to find the similarity.
        - Haversine Distance: To calculate great-circle distance between two points on Earth given latitude and longitude.

3. **Chat Bot:**

    - Use case: Understanding user queries like "How do I request a ride?" or "What do I do if the driver cancels?", and when users ask "How do I set my preferences?" the bot can generate a response like "To set your preferences, go to the profile section and click on 'Preferences.' From there, you can choose your

desired carpool partner preferences." If a user asks, "How do I match with a ride?", the model could generate a step-by-step guide.

- Fine Tuning BERT-based Models (e.g., BERT, DistilBERT) from Hugging Face.
- Using Lang chain to maintain context for conversation.
- **Example Data:** [basicstyle=, `breaklines=true] ["question": "How do I request a ride?", "answer": "To request a ride, tap the request button in the app.", "question": "How can I cancel my ride?", "answer": "You can cancel your ride by tapping the cancel button before the driver arrives.", "question": "What is carpooling?", "answer": "Carpooling is when you share your ride with others going in the same direction."]`

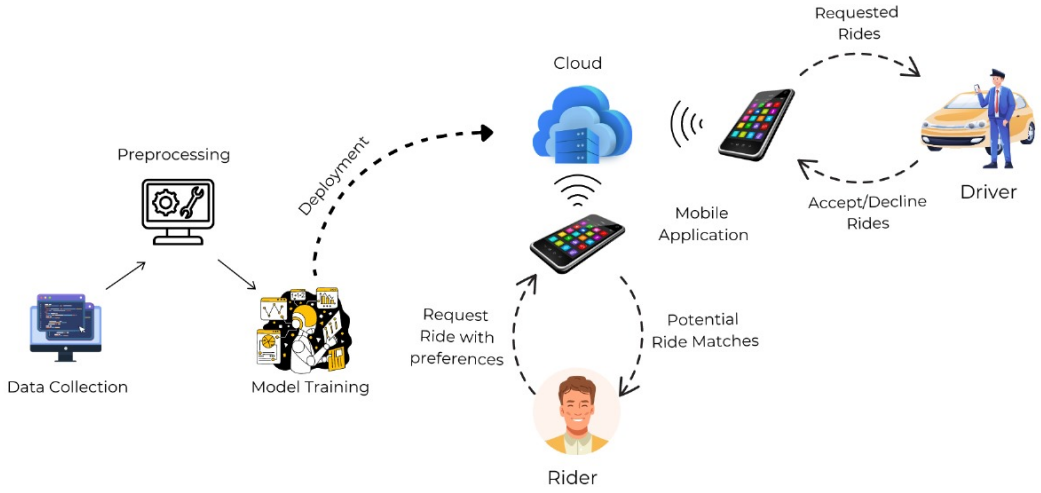

Figure 3.1: Architectural Design

3.2 Design Models

3.2.1 Activity Diagram

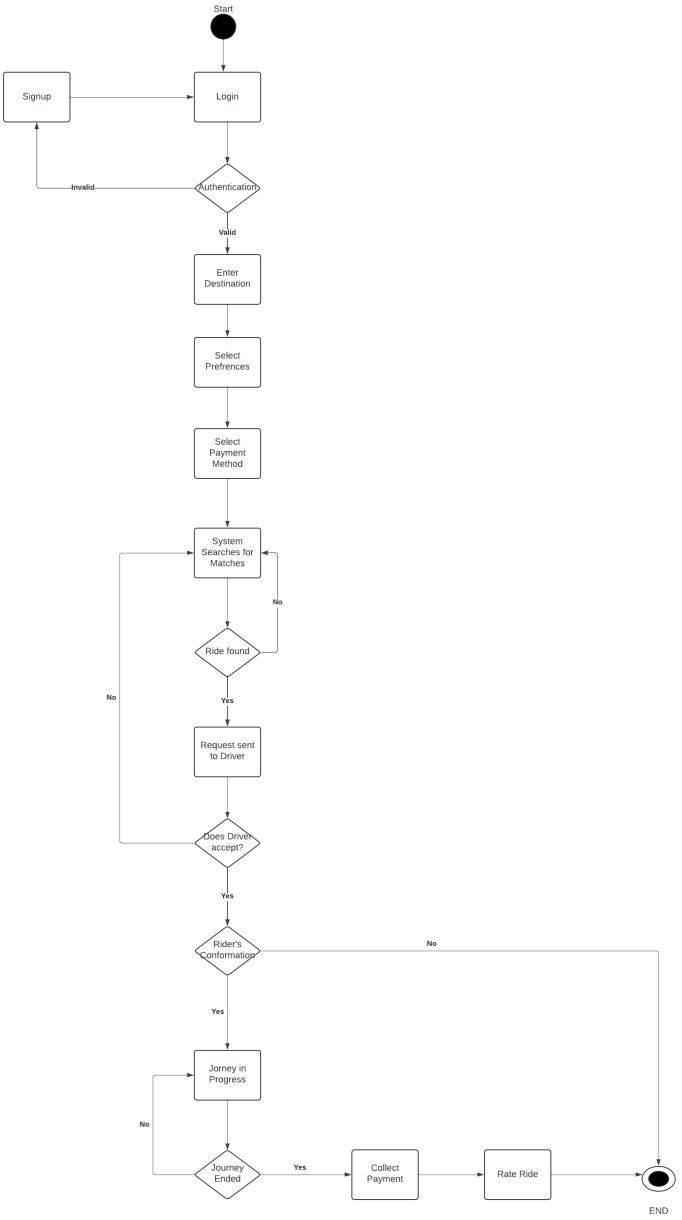

Figure 3.2: Activity Diagram

3.2.2 Class Diagram

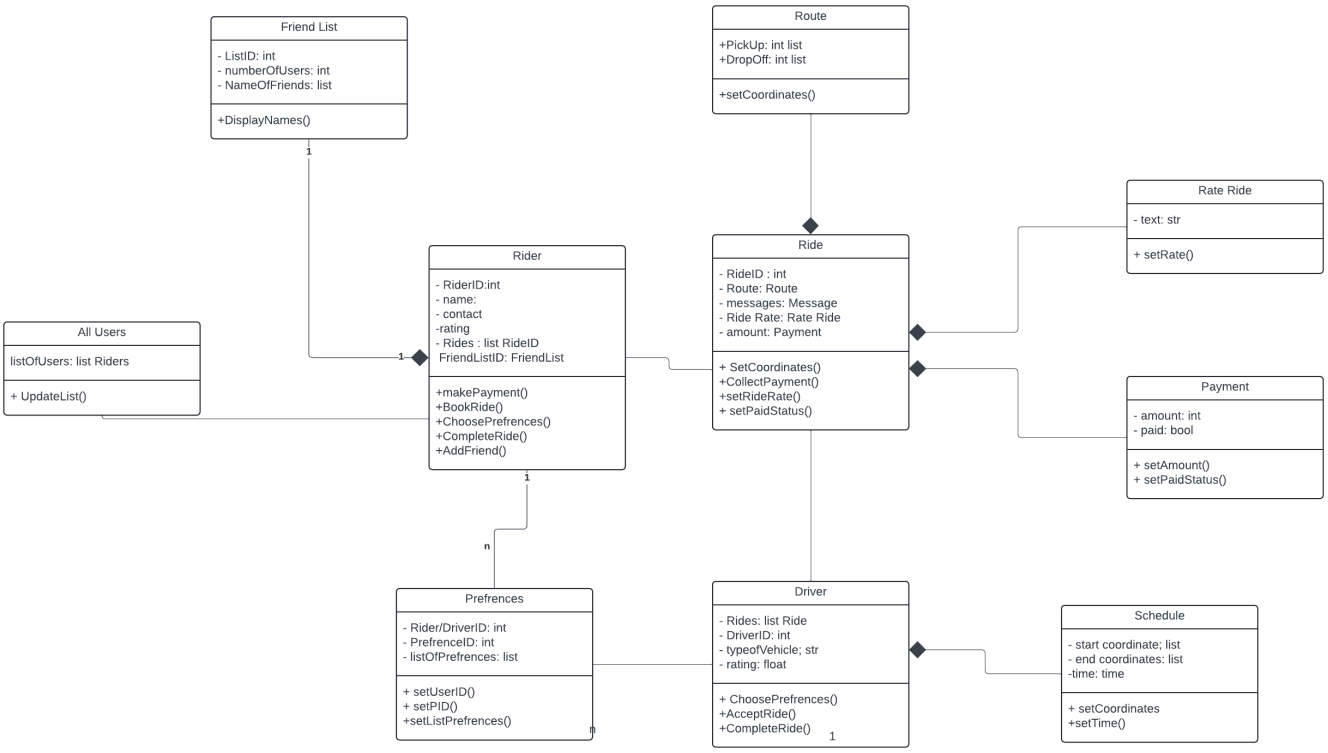

Figure 3.3: Class Diagram

3.2.3 System Sequence Diagram

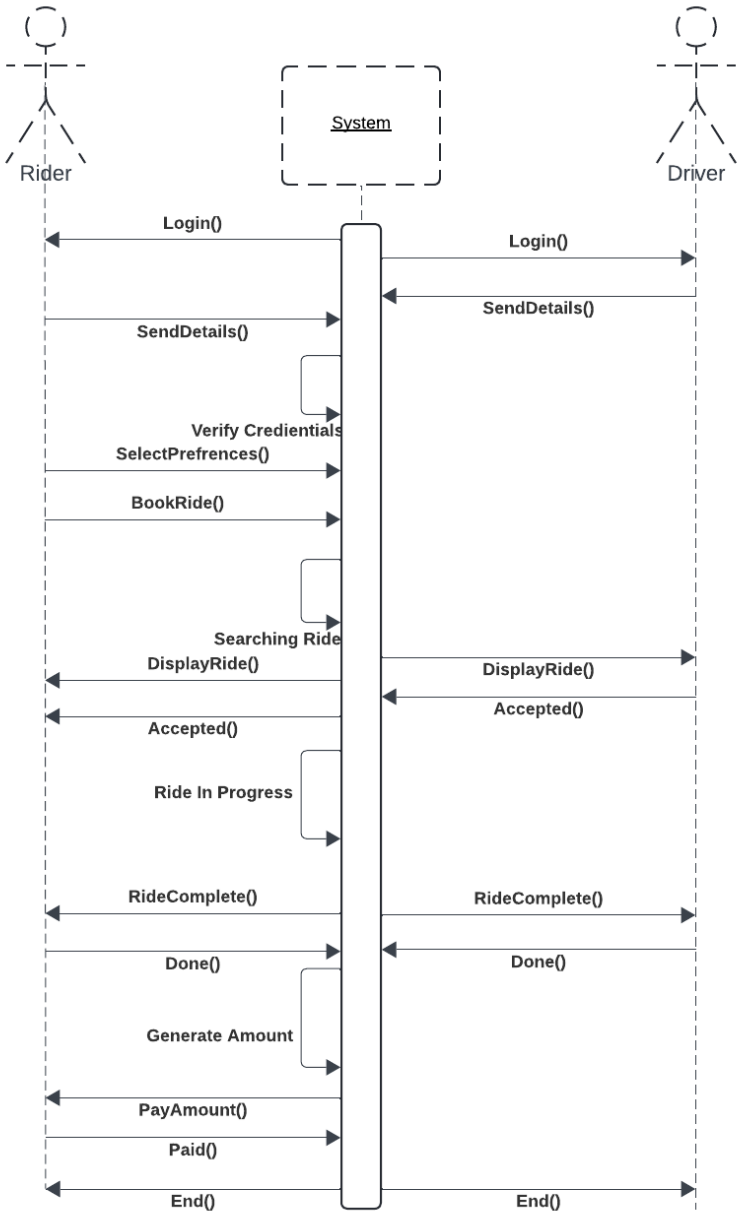

Figure 3.4: System Sequence Diagram

3.3 Data Design

3.3.1 Key Entities

Users: The basic information for users such as `user_id`, `name`, `email`, `phone_number`, and `gender` is stored in this table. Profile data includes social profile links and interests to facilitate social matching. For drivers, additional fields like `vehicle_details` are stored.

Routes: This table contains the route information including `route_id`, `start_location`, `end_location`, `start_time`, `end_time`, and `distance`. This data is critical for matching riders and drivers based on their travel plans.

Rides: The ride table stores the rides created by drivers, with attributes such as `ride_id`, `driver_id`, `available_seats`, `fare`, and `ride_status`. Each ride corresponds to a specific route, detailing the driver and availability of seats for riders.

Ride Matches: This table stores matches between riders and rides based on specific criteria, including route similarity, gender preferences, and social interests. Key fields include `match_id`, `rider_id`, `ride_id`, and `match_criteria`.

Messages: Communication between users (drivers and riders) will be stored here. Attributes include `message_id`, `sender_id`, `receiver_id`, `message_text`, and `timestamp`. This enables users to discuss ride details and confirmations.

Feedback: Feedback and ratings for rides will be collected and stored in this table, helping improve user experience and trust. Attributes include `feedback_id`, `ride_id`, `rider_id`, `rating`, and `comments`.

3.3.2 Data Storage Items

3.3.2.1 Firebase

The primary data storage for user authentication, ride matches, and communication will be handled through Firebase. This service manages real-time data updates, ensuring that users receive instant ride notifications and messages.

3.3.2.2 Google Maps API

The route data, including geolocation and distance, will be fetched via the Google Maps API. Each time a user inputs a route or searches for rides, this data is dynamically queried

and processed. This API also supports real-time navigation and mapping functionalities.

3.3.2.3 Local Databases

Local databases may be used to cache user preferences, recent routes, and previously matched rides for faster access. This enhances the responsiveness of the app when interacting with previously loaded data.

3.3.2.4 Data Flow

The backend (potentially using Firebase) will manage user data and ride matches, while the Google Maps API will provide route-related data in real-time. The message and feedback data will also be stored in Firebase, ensuring secure and fast communication. The local database manages cached data to improve performance, allowing quicker access to frequently used data without requiring constant API calls.

Bibliography